

MORAA GemVision

Work Audit Log · Phase 1 Foundation · generated 2026-10-02

How to use this page: tick the box ☐ at the start of a task once you have checked that it is done and done well. Write the status (done, in progress, blocked, not started) by hand in the empty box at the right of each task, and fill in the summary table below. Nothing on this page is pre-filled with a status. The reasons, decisions (D1, D2...) and errors (E1, E2...) are explained once for the whole phase, after the checklist; tasks refer to them by number.

Domain	Tasks	Done	In progress	Blocked	Not started
1. Test foundation	1				
2. Database schema	1				
3. Dependencies and CI	2				
4. Phase gate (locking)	1				
5. Real-environment checks	2				
6. Lock verification	2				
Total	9				

Why this phase

The backend was built quickly as a prototype and had never been checked on the database it really runs on (PostgreSQL), had no automatic test run, created its database tables in two competing ways, and ran on library versions with known security problems. Money and customer orders depend on all of this. Without a verified foundation, every later fix (wallet ledger, queues, scaling) could break silently. This phase makes the foundation checkable by one command, so a finished phase can be proven and stay locked.

WHAT WAS DONE, IN SHORT

Tests now run against a real PostgreSQL as well as SQLite, GitHub runs them on every change, the database layout is built only by Alembic migrations, the web framework and its security-sensitive libraries were upgraded, and a single 'phase gate' command re-checks everything (tests, image-prompt quality, security advisories, load results, a live boot, read-only production check, staging pooler write test) and writes a report.

Checklist

1. Test foundation

Money paths are tested on the database type used in production.

- ☐
- 1.1 Tests can run on a real PostgreSQL, isolated per test, and refuse to touch the hosted database**
-

FILES backend/tests/db_support.py, backend/tests/test_db_support.py,
backend/tests/test_postgres_semantics.py

HOW TO CHECK From backend, run: set MORAA_TEST_DB=postgres then python -m pytest -q. Expect all tests to pass with none skipped as PostgreSQL-only.

2. Database schema

One owner of the table layout.

☐ 2.1 Tables are created only by migrations, and production refuses to start on a wrong revision

FILES backend/app/database.py, backend/app/main.py, backend/alembic/env.py, backend/tests/test_migrations.py

HOW TO CHECK Run the migration tests; expect all to pass. Starting the app against a database one revision behind in production mode must stop with a clear message.

3. Dependencies and CI

Known-vulnerable libraries upgraded; every change is tested automatically.

☐ 3.1 Framework libraries upgraded and pinned

FILES backend/requirements.txt, backend/constraints.txt, backend/requirements-dev.txt, backend/pyproject.toml, backend/pip-audit-ignore.txt

HOW TO CHECK pip-audit with the ignore list shows no new advisories. Remaining 27 accepted advisories are listed in pip-audit-ignore.txt (mostly Pillow, pending your decision).

☐ 3.2 GitHub runs lint, SQLite tests, PostgreSQL tests and frontend type-check on every change

FILES .github/workflows/ci.yml

HOW TO CHECK Open the pull request on GitHub: both backend and frontend checks show green.

4. Phase gate (locking)

One command proves a phase is intact and records it.

☐ 4.1 Phase gate command, prompt-quality snapshot, load verdict lock, live probes and tests of the gate itself

FILES backend/scripts/phase_gate.py, backend/scripts/prompt_snapshot.py, backend/scripts/live_probe.py, backend/scripts/boot_probe.py, backend/tests/test_phase_gate.py, backend/tests/test_prompt_snapshot.py, backend/tests/snapshots/prompt_hashes.json, backend/tests/load_scenarios/expected_verdicts.json

HOW TO CHECK Run: python scripts/phase_gate.py --phase "Phase 1". The report in docs/phase-gates must say LOCKED; a deliberately broken step must say NOT LOCKED (covered by test_phase_gate.py).

5. Real-environment checks

Behaviour confirmed on the real database and pooler, safely.

☐ 5.1 Read-only startup check against the production database passes without any write

FILES backend/scripts/supabase_readonly_check.py

HOW TO CHECK Run the script; expect all checks OK and no secret printed.

☐ 5.2 Staging pooler write test: 100 simultaneous debits, 20 simultaneous refunds, migrations, all pass

FILES backend/scripts/pooler_write_check.py

HOW TO CHECK With STAGING_DATABASE_URL set in your shell, run the script; expect 'ALL POOLER CHECKS PASSED'. Rotate the staging password first because it was pasted in chat.

6. Lock verification

Phase 1 is proven stable, not just passing once.

☐ 6.1 Full phase gate reports **LOCKED** (one clean run; intermittent PostgreSQL failure hardened, not reproduced)

FILES docs/phase-gates/phase-1-2026-10-02.md, backend/tests/test_postgres_semantics.py, backend/tests/test_refund_once.py, backend/tests/db_support.py

HOW TO CHECK Run `python scripts/phase_gate.py --phase "Phase 1" --live-readonly` from backend. Expect 'GATE: LOCKED'. Run it a few times; any NOT LOCKED report now names the failing tests.

☐ 6.2 GitHub CI confirmed green on the same commit as the final gate report, then tag phase-1-verified

Key decisions and why

D1 Run the PostgreSQL tests on a throw-away local server installed through pip (pgserver), one private schema per test.

WHY THIS CHOICE

SQLite has no row locks, so money races were never truly tested. pgserver needs no Docker or admin rights on Windows and gives each test an isolated schema.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Docker PostgreSQL only** — not chosen because: Not available on this machine; would block local verification.
- **Test against the hosted Supabase database** — not chosen because: Would risk real customer data; a guard now refuses it.

Relates to task(s): 1.1

D2 The database layout is built only by Alembic migrations; the app refuses to start in production if the database revision is behind or unknown.

WHY THIS CHOICE

Two ways of creating tables had already caused drift. One owner of the schema removes that class of bug.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Keep create_all on startup** — not chosen because: Hides missing migrations and drifts from production.

Relates to task(s): 1.2

D3 Use a constraints file and pinned requirements instead of a pip-compile lock.

WHY THIS CHOICE

Simpler to review and works with the existing setup; installs are still reproducible.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **pip-compile lockfile** — not chosen because: Adds a tool and a generated file for no extra safety at this size.

Relates to task(s): 1.3

D4 Upgrade to FastAPI 0.133.1 and Starlette 1.7.0 now, but defer the Pillow 12 upgrade.

WHY THIS CHOICE

The framework upgrade closes known advisories. Pillow decides image output, so upgrading it needs a before/after image check and the owner's decision.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Upgrade Pillow now** — not chosen because: Could silently change generated image quality.

Relates to task(s): 1.3, 1.5

D5 Delete backend/alembic/__init__.py.

WHY THIS CHOICE

It made Python treat the local folder as the Alembic library, so migrations could not be imported.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

None logged.

Relates to task(s): 1.2

D6 Lock image quality with a hash snapshot of all 60 prompt variants.

WHY THIS CHOICE

Prompts decide image quality; any accidental change now fails the gate with no paid calls.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

None logged.

Relates to task(s): 1.5

D7 Treat the pooler 'backend changes' observation as informational only.

WHY THIS CHOICE

An idle pooler can legitimately hand one client the same server connection, so the check was too strict to be a real pass/fail.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

None logged.

Relates to task(s): 1.6

D8 A phase counts as locked only if the gate says LOCKED and GitHub CI is green on the same commit.

WHY THIS CHOICE

Local checks and Linux checks catch different problems; both are needed.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

None logged.

Relates to task(s): 1.5

D9 Gate says LOCKED only if every required step ran and passed; skipping one makes it NOT LOCKED.

WHY THIS CHOICE

A review showed --skip could previously still print LOCKED.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

None logged.

Relates to task(s): 1.7

Errors faced

E1 ImportError: cannot import name 'command' from 'alembic'

Cause: An empty backend/alembic/__init__.py shadowed the Alembic library.

Fix: Deleted the file.

Task: 1.2

E2 CI PostgreSQL step reported green although tests failed

Cause: Output was piped through tee/grep without pipefail.

Fix: Run the step with shell: bash and fail on skipped PostgreSQL tests.

Task: 1.4

E3 Test database guard bypassed with ?host= in the URL

Cause: The guard looked at the host text, not the parsed URL.

Fix: Parse with make_url, reject host/hostaddr/service/passfile keys, check every comma-separated host; tests added.

Task: 1.1

E4 Database newer than code was treated as 'behind'

Cause: Only 'not at head' was checked.

Fix: Added a 'known revision' flag and a clear refusal message.

Task: 1.2

E5 Concurrency tests exhausted the connection pool before the barrier released

Cause: 100 threads each took a connection before waiting.

Fix: Wait at the barrier first, then take a connection.

Task: 1.1

E6 pip: FastAPI 0.133.1 did not move Starlette

Cause: Dependency range allowed the old Starlette.

Fix: Pinned starlette==1.7.0 explicitly.

Task: 1.3

E7 Live probe showed no responses

Cause: Server output pipe was never read and filled up.

Fix: Log to a file instead.

Task: 1.5

E8 Pooler check reported FAIL on a healthy staging database

Cause: My check assumed backends must change between transactions.

Fix: Made it informational.

Task: 1.6

E9 A commit swept in the owner's pre-staged files

Cause: Commit without an explicit path list.

Fix: Soft-reset and recommitted by explicit pathspec; all later commits use pathspecs.

Task: 1.4

E10 PostgreSQL gate step failed twice once, then passed 703 tests on re-run

Cause: Intermittent under machine load; the failing tests were never captured. A static review found short pool/barrier/lock-wait timeouts and silent thread deaths in the concurrency tests.

Fix: Pool timeout 15s to 120s, barriers 20-30s to 120s, lock-wait query narrowed, thread errors collected and asserted, schema drop gets a lock timeout. Gate also now fails if a required step is skipped. Two later full PostgreSQL runs and one gate run passed; cannot prove the flake is gone, only that it did not recur.

Task: 1.7